PROJECT REPORT

ClaimGuard

Fraud triage for vehicle insurance claims

Design decisions · architecture · reasoning · outcomes | 13 November 2026 | model hgb-v2

What it is

ClaimGuard scores an incoming vehicle insurance claim for fraud risk and places it in one of three review bands: **LOW** (auto-approve), **MEDIUM** (quick review by a claims handler) or **HIGH** (send to the investigation team). Every decision comes with the red-flag rules it hit, the top reasons from the model in plain English, and a tamper-evident audit record. Investigators record what they found, and the system reports how often each band turns out to be real fraud.

Design principle: ClaimGuard ranks claims for human review. It never rejects a claim on its own. Every choice below follows from that: calibrated probabilities instead of raw scores, explanations on every decision, an audit trail, and bands sized to what a review team can handle.

Key outcomes

	Result	Meaning
Ranking quality on an unseen year	PR-AUC 0.160 (base rate 0.052)	about 3x better than random at putting fraud near the top
Gain from training on more years	+0.023 PR-AUC, 95% CI [+0.005, +0.043]	paired bootstrap on 1996; better in 99.2% of resamples
Review policy on 1996 (1994 model)	29% reviewed, 57% of fraud caught	cost 0.485 per claim vs 0.522 reviewing nothing (10:1 ratio)
Shipped model	14,345 claims, 817 fraud	1994-1996, CI holdout excluded; 2x the fraud examples of 1994 alone
Batch scoring speed	1,074 claims in about 1 s	was 33 s scoring one claim at a time
Quality gates	68 automated tests, CI, Docker	model-quality floor on a holdout it never saw

1. The problem and its constraints

The data is the public *Vehicle Insurance Claim Fraud Detection* set (fraud_oracle.csv): 15,419 claims from 1994 to 1996 with 31 columns, almost all of them categories or ranges, and no claim amounts. Five facts about this problem shaped the design:

- **Fraud is rare (about 6%).** Accuracy is meaningless: predicting 'honest' for everything scores 94%. Models are compared on **PR-AUC** (average precision), which only rewards putting fraud near the top.
- **Patterns drift over time.** A model is used on claims that arrive after it was trained, so it must be judged the same way: train on earlier years, test on a later one. Random splits overstate quality.
- **People act on the output.** An investigator needs to know *why* a claim was flagged, and a probability needs to mean what it says, so thresholds can be set from costs.
- **Decisions must be defensible.** Insurance decisions get disputed and audited, so every score must be traceable to the exact claim, model version and configuration that produced it.
- **Fairness and regulation.** Several regulators forbid using sex in insurance decisions, and age-based decisions draw scrutiny. The cost of excluding them has to be measured, not assumed.

2. Architecture

Two halves share one Python package. The **offline** half (train.py) makes every modelling decision on the data and writes a single model bundle plus human-readable reports. The **online** half (a FastAPI service) loads that bundle once and scores claims. Business assumptions (rules, reason texts, cost ratio, team capacity) live in YAML files under config/, not in code.

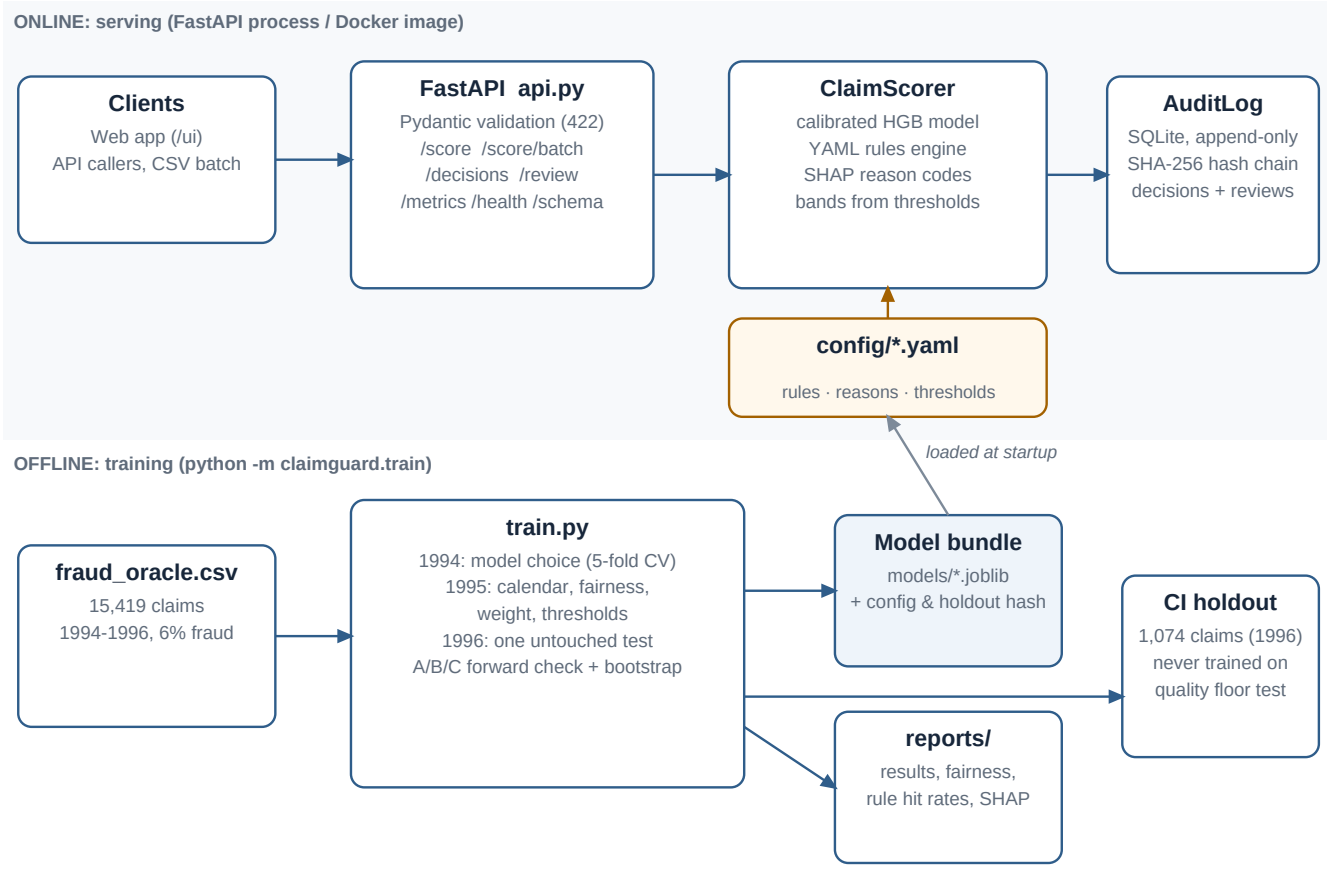


Figure 1. Components and data flow. Arrows show what feeds what.

Request flow for one claim

- **Validate.** A Pydantic model built from the shared vocabulary in schema.py accepts only known categories and in-range numbers. Anything else is a 422 error, never a silently wrong score.

- **Score.** The calibrated gradient-boosting model gives a fraud probability. The rules engine evaluates the red flags. Combined score = w x model + (1 - w) x rules, with w chosen on validation data.
- **Band.** Compare with the MEDIUM and HIGH thresholds; a claim hitting a 'strong' rule is lifted to at least MEDIUM.
- **Explain.** SHAP values for the claim are summed back to the original columns and written as sentences.
- **Log.** The claim, score, band, rules, reasons, model version and config hash are appended to the audit chain, and the decision ID is returned.

Module map

Module	Responsibility
schema.py	One vocabulary for every column: allowed values, order of ranges, numeric limits. The encoder, rules and API all read it, so they cannot disagree.
data.py / features.py	Load and clean the CSV (drop ID columns, unknown age 0 -> missing); ordinal-encode ranges, one-hot the rest.
model.py	Model candidates, calibration, imbalanced-data metrics, bundle save/load.
rules.py	YAML rules engine, validated at load time; config hash.
explain.py	SHAP reason codes grouped per raw column, batched for many claims.
scoring.py	Combine, band assignment, cost and capacity thresholds; ClaimScorer used by the API.
audit.py	Append-only, hash-chained SQLite log; review queue and metrics queries; verify CLI.
api.py + static/	REST API and the web app (plain HTML/CSS/JS served by FastAPI).
train.py	The full experiment: every decision in section 3, reports, bundle, CI holdout.

3. Modelling decisions

Every decision is made on data the final test never touches: **1994 is for training** (with 5-fold stratified cross-validation inside it), **1995 is for choosing settings**, and **1996 is tested once**. train.py runs these steps in order and writes the evidence to reports/results.md.

Decision	Chosen	Why, and the evidence
Evaluation split	By year: 1994 / 1995 / 1996	Matches real use (score future claims). Random splits mix claims from the same weeks and overstate quality.
Main metric	PR-AUC	Fraud is 5-7%. PR-AUC rewards ranking fraud at the top; ROC-AUC and accuracy look good even for weak models.
Calendar columns (month, weekday, week)	Dropped	Cross-validation inside 1994 said they help (PR-AUC 0.274 vs 0.245), but on 1995 they hurt badly (0.117 vs 0.186). They memorise one year's patterns.
ID columns (policy number, rep number)	Dropped	No real signal, and they let a model memorise rows.
Algorithm	HistGradientBoosting, tuned	5-fold CV PR-AUC on 1994: logistic regression 0.208, random forest 0.240, boosting 0.245, tuned boosting 0.287 (40-candidate random search).
Class imbalance	Class weights, not SMOTE	SMOTE applied correctly inside each fold scored 0.253 vs 0.287. Weights also keep the model trained on real claims, which SHAP then explains.
Calibration	Sigmoid (Platt), out-of-fold, ensemble=False	Class weights inflate scores (mean 0.31 vs a 5.8% fraud rate). Calibrated mean 0.062, Brier 0.050 vs 0.167. Sigmoid over isotonic: only hundreds of fraud cases. ensemble=False leaves one model for SHAP.
Sex, Age, AgeOfPolicyHolder	Excluded (policy)	Including them adds 0.026 validation PR-AUC, but the model then flags women at about half the rate of men and catches far less of their fraud. Treated as policy, with the cost reported in fairness.md.
Rules in the score	Weight 1.0 on the model	Searched w in 0.5-1.0 on 1995: pure model ranked best (0.192). Rules still set a minimum band and are shown as reasons.
Which rules stay on	4 of 7	Kept only if the claims they flag are at least 1.5x as likely to be fraud as average in 1994. 'Strong' only if that fraud share beats the cost break-even 1/(1+10) = 9.1%.
MEDIUM threshold	Cost-minimising	Assumed: a missed fraud costs 10 reviews. With calibrated scores this lands near the break-even probability (0.094 on 1995).
HIGH threshold	Top 5% of scores	Sized to the investigation team's assumed capacity, not to a statistic.

All business assumptions (cost ratio 10:1, HIGH capacity 5%, rule weights) are marked as assumptions in config/thresholds.yaml. Changing them and retraining recomputes the thresholds; serving refuses to hide a mismatch (see section 6).

4. Training the shipped model on all three years

The first version shipped a model trained only on 1994 (6,141 claims, 409 fraud), because 1995 set the thresholds and 1996 was the test. That leaves two thirds of the data unused. The question was whether more data helps, without losing an honest estimate of performance.

How it was answered

- The selection pipeline above is unchanged and still reports its untouched 1996 test.
- **New threshold method.** If 1995 becomes training data, there is no spare year for thresholds. Instead they are set from **out-of-fold** calibrated scores: 5-fold CV over the training claims, so each score comes from a model that never saw that claim.
- **Forward check on 1996**, using only models that never saw 1996:

Variant	Trained on	PR-AUC	Reviewed	Fraud caught	Cost/claim
A: thresholds from 1995	1994	0.136	29%	57%	0.485
B: out-of-fold thresholds	1994	0.136	35%	69%	0.474
C: out-of-fold thresholds	1994 + 1995	0.160	23%	48%	0.476

- **A vs B**: the new threshold method is as good as a separate year (slightly lower cost), so it is safe to use.
- **A vs C**: adding 1995 raises 1996 PR-AUC from 0.136 to 0.160. 1996 has only 213 fraud cases, so a paired bootstrap (2,000 resamples of the same claims) checked it: gain +0.023, 95% CI [+0.005, +0.043], C better in 99.2% of resamples. The gain is real, not noise.
- C reviews fewer claims than A because its thresholds came out higher; cost per claim is about the same.

What ships

The model is refit with the same settings on **14,345 claims from 1994-1996 (817 fraud, double the 409 in 1994)**. Thresholds from out-of-fold scores: MEDIUM >= 0.0977, HIGH >= 0.1743. Its best estimate for new claims is row C. The 1,074-claim CI holdout from 1996 is kept out of training, and the shipped model scores PR-AUC 0.224 on it (holdout fraud rate 9.9%). That number guards against regressions; it is somewhat optimistic because the holdout shares a year with training data. The choice is one switch: final_fit in config/thresholds.yaml.

5. Explainability and rules

Each decision lists the top three features that pushed *that* claim towards fraud. SHAP values from the tree model are computed on the encoded features, then one-hot pieces are summed back to the original column, so an investigator sees 'Policy holder was at fault', not 'Fault_Policy Holder = 1'. Texts come from config/reasons.yaml and can be edited without code. Calibration is a monotone map, so it does not change the ranking of reasons.

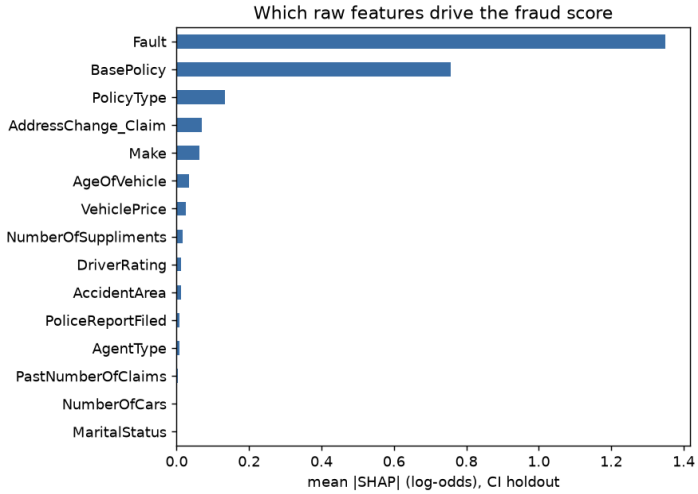


Figure 2. What drives the shipped model (mean |SHAP| on the CI holdout). Who was at fault and the policy type dominate, which matches how this fraud tends to appear in the data.

Rules engine

Rules are hand-written red flags in config/rules.yaml (for example 'accident within 15 days of the policy starting'). They are validated when loaded: an unknown field or a misspelled value stops the service at startup instead of silently never firing. Each rule was measured on 1994 and checked on 1995 (reports/rule_hit_rates.md). Three intuitive rules were switched off because the data contradicted them: 'no police report and no witness' fires on 97% of claims; 'frequent claimant' and 'many supplements' flag claims that are *less* likely than average to be fraud.

6. Audit trail and governance

- **Append-only.** SQLite triggers reject every UPDATE and DELETE on decisions and reviews. Investigator outcomes are new rows in a separate table; the latest one counts in the metrics.
- **Tamper-evident.** Each row stores the SHA-256 of its own contents and the previous row's hash. `python -m claimguard.audit verify` reports any changed, removed or reordered row. Writes take a write lock so the chain cannot fork. Known limit: someone with file access could rebuild the whole chain, so the latest hash (audit head) should be published elsewhere.
- **Reproducible decisions.** Every decision stores the model version and a fingerprint of the three config files. /health reports config_matches_training: false if someone edits the rules or thresholds without retraining. The fingerprint ignores line endings so a Windows checkout matches Linux.
- **Model and holdout are tied together.** The bundle stores the hash of the CI holdout it excluded, and a test checks it, so a future retrain cannot quietly train on the claims used to judge it.
- **Monitoring.** /metrics (and the web app's Monitoring tab) shows the confirmed-fraud rate per band from real outcomes. If HIGH stops beating LOW, the model needs attention.

7. Service and web app

Endpoint	Purpose
POST /score	Score one claim (JSON). Optional claim_ref is stored and echoed back.
POST /score/batch	Score a CSV of up to 5,000 claims. Bad rows are reported, good rows still scored.
GET /decisions	Review queue: filter by band and review status, sort by score or recency.
GET /decisions/{id} · POST .../review	Full decision with reviews; record CONFIRMED_FRAUD or CLEARED.
GET /metrics · /health · /schema	Outcomes per band; model, data and config status; form definition.

The web app is served by the same process at /. It has four tabs: score a claim, batch upload, review queue, and monitoring. It is plain HTML, CSS and JavaScript with no build step, so the Docker image and CI need nothing extra. The form is generated from /schema, which reads schema.py, so it can only offer values the API accepts; fields the model does not use are greyed out. All user-supplied text is escaped before display.

Performance

Testing the running service showed batch scoring at 31 ms per claim (1,074 claims in 33 s), which would take about 2.5 minutes at the 5,000-row limit and risk HTTP timeouts. Profiling split the time into the model call (11 ms), SHAP (10 ms) and one audit transaction per claim (3 ms). Batches now make one model call and one SHAP call for all rows and write the audit rows in a single transaction: **about 1 second for 1,074 claims**. Single-claim scoring uses the same code path, and a test asserts that batch results equal one-at-a-time results exactly.

8. Quality and operations

- **68 automated tests**: scoring maths, rules validation, audit chain and tamper detection, every API endpoint including error cases, the web app routes, and a model-quality floor (PR-AUC >= 0.17, ROC-AUC >= 0.70, calibrated mean score) on the fixed holdout.
- **CI** (GitHub Actions) runs the tests against the committed model, then builds the Docker image and checks /health in the running container.

- **Docker**: slim Python 3.13 image, non-root user, audit database on a volume, health check.
- **Pinned dependencies**: the model is a pickle and must load with the same scikit-learn version.
- **End-to-end test** of the live server: health, low- and high-risk claims, validation errors, the full holdout batch, review, lookup, metrics, and audit verification (1,076 decisions, chain intact).

Defects found and fixed during review

Defect	Effect	Fix
Blank age rejected in batches	Rows with unrecorded age (blank cell) were refused	Age optional; blank, missing and 0 all mean 'not recorded'
Numbers stored as text in the audit log	13% of reasons were stored as "500", not 500, so the record differed from the response	Convert numpy values to plain Python types
Line endings changed the config fingerprint	A Windows checkout would report the config as changed and fail CI	Fingerprint ignores CRLF/LF; .gitattributes keeps LF
claim_ref never returned	Batch callers could only match results by row number	Echoed in every decision
Slow batch scoring	33 s for 1,074 claims	Vectorised scoring and one audit transaction (about 1 s)

9. Limitations and next steps

- **Modest signal.** PR-AUC of 0.14-0.16 on an unseen year is real but limited: the data has no claim amounts, no text, no network of related parties. This is why ClaimGuard only prioritises review.
- **Old, public data (1994-1996).** A real deployment must retrain on the insurer's own recent claims and re-run every check in this report.
- **Assumed costs.** The 10:1 cost ratio and 5% HIGH capacity are placeholders. With claim amounts, the cost model should use money, and thresholds could depend on claim size.
- **No later year for the shipped model.** Row C of section 4 is the best estimate; the confirmed-fraud rates in /metrics are the real test once outcomes arrive.
- **Feedback is biased.** Outcomes only exist for reviewed claims (mostly MEDIUM/HIGH). Reviewing a small random sample of LOW claims would measure missed fraud and give unbiased labels for retraining.
- **Single-node storage.** SQLite suits one service instance. Several instances would need a shared database (for example PostgreSQL) with the same append-only rules, and authentication for the API and app.

Reproducing everything

`pip install -r requirements-dev.txt` then `pytest -q` checks the committed model. With fraud_oracle.csv in data/raw/, `python -m claimguard.train` reruns every experiment in this report with fixed seeds and regenerates the model, reports/results.md, reports/fairness.md and reports/rule_hit_rates.md. `uvicorn claimguard.api:app` starts the service and the web app.